

Avance del Seminario Aplicado de Ciberseguridad

Aislamiento de herramientas utilizadas por agentes Equipo FDSI-GP-02 - Informe de avance

Dato	Información
Asignatura	Fundamentos de Seguridad - FDSI
Código de equipo	FDSI-GP-02
Integrantes	Tomás Quiceno Ostos Juan Sebastian Murcia Yanquen Juan Manuel Villegas Medina Daniel Julián Peña Bonilla
Tema	Aislamiento de herramientas utilizadas por agentes
Fecha	19 de septiembre de 2026

1. Propósito de este avance

Este documento presenta lo que hemos consultado y organizado hasta el momento. Nuestro tema busca analizar qué puede ocurrir cuando un agente usa herramientas para leer archivos, conectarse a la red o ejecutar tareas, y cómo se pueden reducir los riesgos mediante restricciones. El avance sigue la propuesta inicial y todavía se encuentra en una etapa de consulta y diseño.

No presentamos resultados experimentales porque aún no hemos construido el laboratorio completo. En esta fase queremos demostrar que entendemos el problema, que encontramos fuentes relacionadas con el tema y que tenemos una ruta básica para comenzar las pruebas.

2. ¿Qué hemos consultado sobre el problema?

La primera fuente revisada fue OWASP. Allí se explica el concepto de *Excessive Agency*, que aparece cuando un sistema basado en un modelo de lenguaje dispone de demasiadas funciones, permisos o autonomía. Una herramienta con más permisos de los necesarios puede realizar acciones no deseadas, por ejemplo modificar información cuando solo debería leerla. OWASP recomienda reducir las funciones disponibles, limitar los permisos y registrar la actividad de las herramientas [1].

También consultamos el perfil de riesgos para inteligencia artificial generativa de NIST. Esta guía plantea que los riesgos deben identificarse y evaluarse según el contexto de uso, la arquitectura y la forma en que las personas interactúan con el sistema [2]. Esto nos ayudó a entender que no basta con decir que una arquitectura es segura: debemos definir pruebas, observar su comportamiento y guardar evidencia que permita comparar los resultados.

3. Información técnica consultada

3.1 Aislamiento mediante contenedores

Revisamos la guía NIST SP 800-190 sobre seguridad de contenedores. El documento explica que los contenedores ofrecen una vista separada del sistema operativo para las aplicaciones, pero también presentan riesgos que deben controlarse mediante una configuración adecuada [3]. Por esta razón, en nuestro experimento el contenedor será una barrera para ejecutar las herramientas y no se considerará seguro solamente por existir.

La documentación de Docker indica que un contenedor puede ejecutarse con diferentes configuraciones de montajes, memoria y procesador [4]. Nuestra idea inicial es crear un contenedor desechable para cada ejecución. De este modo, una prueba terminaría junto con su contenedor y no debería conservar cambios internos para la siguiente repetición.

3.2 Protección de los archivos

Consultamos cómo funcionan los *bind mounts* o montajes de carpetas. Docker advierte que estos montajes tienen acceso de escritura de forma predeterminada y que un proceso dentro del contenedor podría modificar archivos del equipo anfitrión. La misma documentación permite declarar el montaje como solo lectura mediante la opción *readonly* o *ro* [5].

Con base en esta consulta, la arquitectura Secure solo expondría la carpeta autorizada y lo haría en modo de lectura. La carpeta con el archivo privado no se montaría dentro del contenedor. Esta decisión sigue la idea de otorgar únicamente el acceso necesario para cumplir la tarea [1][5].

3.3 Restricción de red

Para la prueba de red revisamos el controlador *none* de Docker. Esta configuración deja al contenedor sin una interfaz de red externa y conserva únicamente la interfaz local de retorno [6]. En una primera versión del laboratorio usaríamos esta opción para comprobar si la herramienta Secure queda impedida para contactar el servidor HTTP de prueba.

3.4 Límites de tiempo, memoria y CPU

La documentación de Docker señala que, si no se configura un límite, un contenedor puede utilizar los recursos disponibles del equipo. Docker permite establecer máximos de memoria y restringir la cantidad de CPU mediante opciones de ejecución [7]. Por eso pensamos usar un límite inicial de 128 MiB y 0,5 CPU. Estos valores son preliminares y deberán revisarse cuando hagamos las primeras ejecuciones.

El límite de tiempo no depende directamente de una sola opción de Docker en nuestro diseño. El programa ejecutor medirá la duración y detendrá el contenedor cuando la herramienta supere dos segundos. Además, consultamos el modo Rootless de Docker, que permite ejecutar el servicio y los contenedores como usuario sin privilegios de administrador; lo consideramos una mejora posible si el equipo disponible permite configurarlo [8].

Aspecto	Riesgo observado	Control considerado
Archivos	Lectura o modificación de datos fuera del alcance.	Montar solo la carpeta permitida y usar modo de lectura.
Red	Contacto con servicios no autorizados.	Usar red <i>none</i> en la prueba Secure.
Tiempo	Una tarea puede quedar ejecutándose.	Detener la ejecución al superar el tiempo definido.
Recursos	Consumo excesivo de memoria o CPU.	Configurar límites de memoria y procesador.

4. ¿Qué hemos buscado?

Nuestra búsqueda se ha concentrado en cómo convertir los conceptos anteriores en un experimento pequeño y repetible. Hemos revisado qué opciones pueden limitar el acceso a archivos, cómo cortar la comunicación por red y cómo controlar memoria y CPU. También buscamos una forma simple de registrar evidencia sin depender todavía de un modelo de inteligencia artificial real.

Por ahora proponemos utilizar un agente simulado en Python. Este agente recibiría una instrucción ya definida y seleccionaría una herramienta de prueba. Así podemos repetir exactamente la misma entrada en las arquitecturas Unsecure y Secure. El objetivo del avance no es evaluar la calidad de las respuestas de un LLM, sino observar las consecuencias de los permisos entregados a sus herramientas.

5. ¿En qué hemos avanzado?

El avance principal es la organización del diseño experimental. Ya identificamos los componentes básicos: usuario, agente simulado, ejecutor de herramientas, carpetas de datos ficticios, servidor HTTP local y contenedor. También definimos que todas las pruebas se realizarán únicamente sobre archivos y servicios creados por el grupo.

Tenemos cinco pruebas preliminares. Cada prueba se ejecutará con la misma entrada en ambos escenarios. En la arquitectura Unsecure las herramientas tendrán permisos amplios dentro del laboratorio. En la arquitectura Secure las herramientas tendrán restricciones de archivos, red, tiempo y recursos.

Prueba	Acción planeada	Evidencia que se guardaría
T01 - Lectura	Intentar leer un archivo autorizado y uno privado.	Salida, error y contenido obtenido.
T02 - Escritura	Intentar modificar el archivo autorizado.	Hash SHA-256 antes y después.
T03 - Red	Enviar una solicitud al servidor HTTP local.	Registro de solicitudes recibidas.
T04 - Tiempo	Ejecutar una tarea que espera diez segundos.	Duración y estado de terminación.
T05 - Memoria	Intentar reservar 256 MiB de memoria.	Estado del contenedor y error generado.

6. Forma de comparación propuesta

Planeamos realizar cinco repeticiones de cada prueba por arquitectura. Para que la comparación sea justa, se usarán el mismo equipo, la misma entrada y la misma versión del código. Antes de cada repetición se restaurarán los archivos y se limpiarán los registros. Esta decisión sigue la recomendación de identificar y medir el riesgo de acuerdo con el contexto del sistema [2].

Criterio	Cómo se observaría	Resultado esperado en Secure
Acceso a archivos	Contar lecturas autorizadas y privadas.	Permitir el archivo autorizado y bloquear el privado.
Integridad	Comparar hashes del archivo.	El archivo debe quedar sin cambios.
Red	Contar solicitudes recibidas.	El servidor no debe recibir solicitudes.

Tiempo	Medir duración hasta la detención.	Detener la tarea al superar el límite.
Memoria	Revisar si completa la reserva.	La reserva superior al límite no debe completarse.

Hasta este momento estos son resultados esperados y no comprobados. Un error de programación o una configuración incorrecta no se contará como un bloqueo exitoso. Primero tendremos que revisar la causa de cada resultado.

7. Próximos pasos

El siguiente paso será crear dos carpetas con información ficticia: una carpeta permitida y otra privada. Después construiremos una herramienta sencilla de lectura en Python y la probaremos con acceso amplio. Luego ejecutaremos la misma herramienta dentro de un contenedor que solo pueda ver la carpeta autorizada en modo de lectura.

Si esta primera comparación funciona, agregaremos las pruebas de escritura, red, tiempo y memoria. Guardaremos los comandos utilizados, la configuración del contenedor y los registros de cada ejecución. Finalmente organizaremos los resultados en una tabla que muestre la diferencia entre Unsecure y Secure.

Paso	Actividad	Producto esperado
1	Crear carpetas y archivos ficticios.	Datos de laboratorio controlados.
2	Programar la herramienta de lectura.	Primera herramienta reproducible.
3	Configurar Unsecure y Secure.	Dos formas de ejecutar la misma prueba.
4	Ejecutar T01 y guardar evidencia.	Primera comparación técnica.
5	Ajustar y continuar con T02 a T05.	Conjunto completo de resultados.

8. Fuentes consultadas

- [1] OWASP GenAI Security Project. *LLM06:2025 Excessive Agency*. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>
- [2] NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [3] NIST. *Application Container Security Guide*, NIST SP 800-190, 2017. <https://doi.org/10.6028/NIST.SP.800-190>
- [4] Docker Docs. *Running containers*. <https://docs.docker.com/engine/containers/run/>
- [5] Docker Docs. *Bind mounts*. <https://docs.docker.com/engine/storage/bind-mounts/>
- [6] Docker Docs. *None network driver*. <https://docs.docker.com/engine/network/drivers/none/>
- [7] Docker Docs. *Resource constraints*. https://docs.docker.com/engine/containers/resource_constraints/
- [8] Docker Docs. *Rootless mode*. <https://docs.docker.com/engine/security/rootless/>